Awarding Body : Arden University
Programme Name : Data Visualisation - COM7021
Module Name (and Part if applicable) :
Assessment Title : COM7021 Assessment
Student Number : 25290556
Tutor Name : Ahmed Hassan
Word Count : 2333

Contents

List of Figure and Tables 3
Task 1 Data Visualisation in Python 4
1.1. Scope and Design 4
1.2. Visual Findings 4
1.3. Interactive Dashboard 9
Task 2 Data Visualisation Report....11
2.1. Critical Findings 11
2.2. Accessibility and Wider Implications....12
2.3. Dashboard Evaluation 13
2.4. Recommendations and Conclusion....13
References....14
Appendix 15

List of Figure and Tables

Table 1: Dataset summary....4
Table 2: Key findings....11

Figure 1: Monthly passenger demand....5
Figure 2: Passenger demand by city....6
Figure 3: Delay distribution by weather and event day....7
Figure 4: Performance metric correlations....7
Figure 5: Revenue and cost by transport mode....8
Figure 6: Accessibility and sustainability by transport mode....9
Figure 7: Full interactive dashboard....10
Figure 8: Filtered dashboard for Germany and Metro....10
Figure 9: Monthly demand code....15
Figure 10: City demand code....15
Figure 11: Delay distribution code part 1....16
Figure 12: Delay distribution code part 2....17
Figure 13: Correlation code part 1....18
Figure 14: Correlation code part 2....18
Figure 15: Financial performance code....19
Figure 16: Accessibility and sustainability code part 1....20
Figure 17: Accessibility and sustainability code part 2....20
Figure 18: Dashboard filters code....21
Figure 19: Dashboard KPI code....22
Figure 20: Dashboard interactive views code....22

Task 1 Data Visualisation in Python

1.1. Scope and Design

The investigation period covered 5000 records of the EcoMove database between January 2024 and December 2025, and examined the data using Python. The 18 fields that were included encompassed demand, reliability, finance, customer experience, accessibility and environmental performance. There were no missing values, no duplicate record IDs, and no invalid dates in the checks and they were all included in the analysis. Visual analytics can support users to navigate and explore complex mobility data in order to uncover decision patterns (Delfini, Spahiu and Vizzari, 2024). Six visualisations were used to answer six different management questions. The labels were clear, familiar units and limited visual complexity helped to facilitate readability. Values were in the European style and hatching made the use of colour less important.

Table 1: Dataset summary

Item	Value
Records	5,000
Fields	18
Period	Jan 2024 to Dec 2025
Cities	12
Countries	6
Modes	6
Routes	6
Weather groups	6
Missing values	0
Duplicate IDs	0

1.2. Visual Findings

The monthly demand fluctuated in the two years. The biggest number of passengers was in March 2024 with 2,080,100 passengers, and the smallest number of passengers was in October 2025 with 1,409,837 passengers. A line graph maintained the order of time and was easy to use for indicating changes from month to month. Demand for the city also was uneven.

Paris recorded 4,492,481 passengers and Berlin 4,487,351, while Lyon recorded 2,536,528. A horizontal bar chart was used, with the bars ranked by height, making the difference between the cities easy to compare. Operational data can be transformed into useful decision information using transport dashboards and visual repositories (Tsouros et al., 2024). The line chart was also found to allow seasonal changes to be seen without obscuring the individual months. The same scale was used on the city chart so that the managers could be compared without deciphering the different axis or categories.

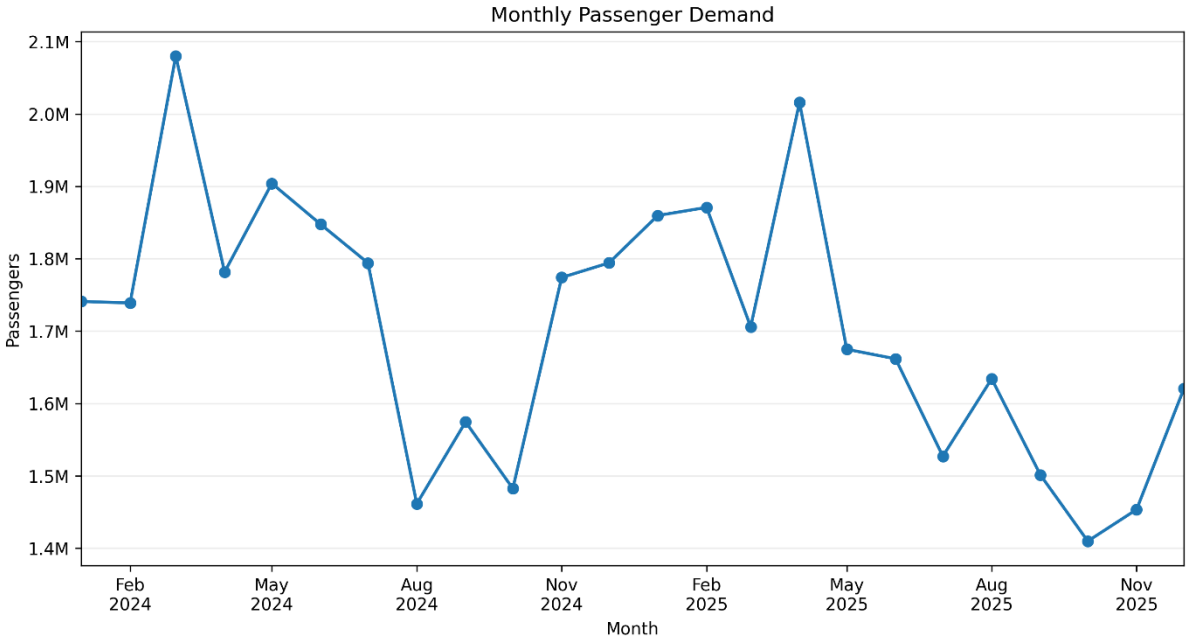


Figure 1: Monthly passenger demand

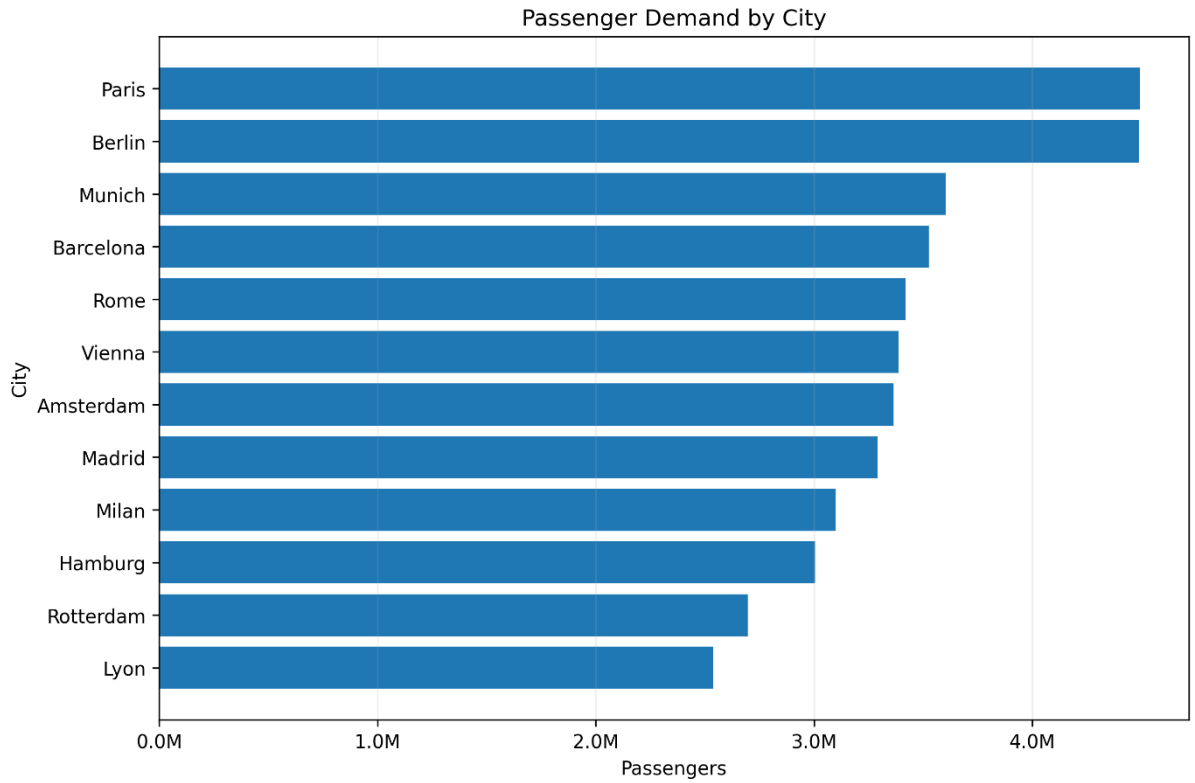


Figure 2: Passenger demand by city

Reliability differences were observed on weather and event days. The average delay for snow event days was the highest at 9.01 minutes. Clear normal days with the lowest at 3.58 minutes. Average delay for each weather category was greater during event days than during non-event days. The box plot was used to display both the spread and the central values. One of the critical service quality indicators for public transport assessment is reliability (Institute for Transportation and Development Policy, 2024). The heatmap highlighted that there were strong relationships between the customer rating and delay and between the customer rating and on-time performance. The correlation for delay was found to be minus 0.97 and for rating minus 0.82. These are associations and not evidence for causation. This was useful because average delay may not show any variation in each of the weather groups. Displaying distributions provided managers with better insight into normal operating conditions and incidents that occurred outside of the norm.

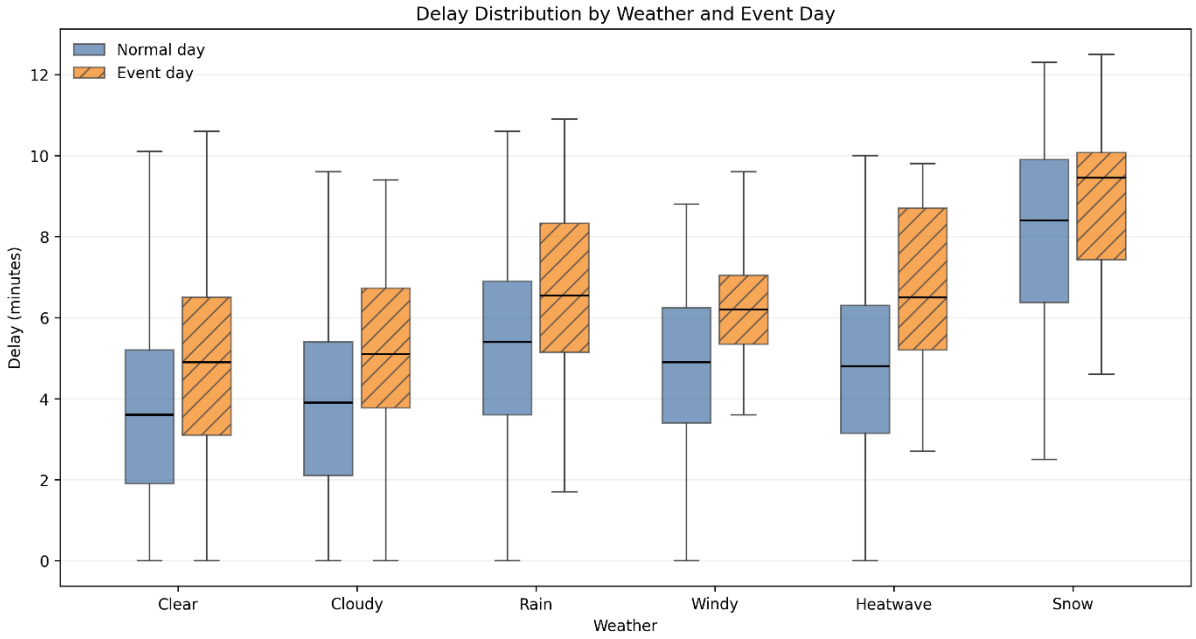


Figure 3: Delay distribution by weather and event day

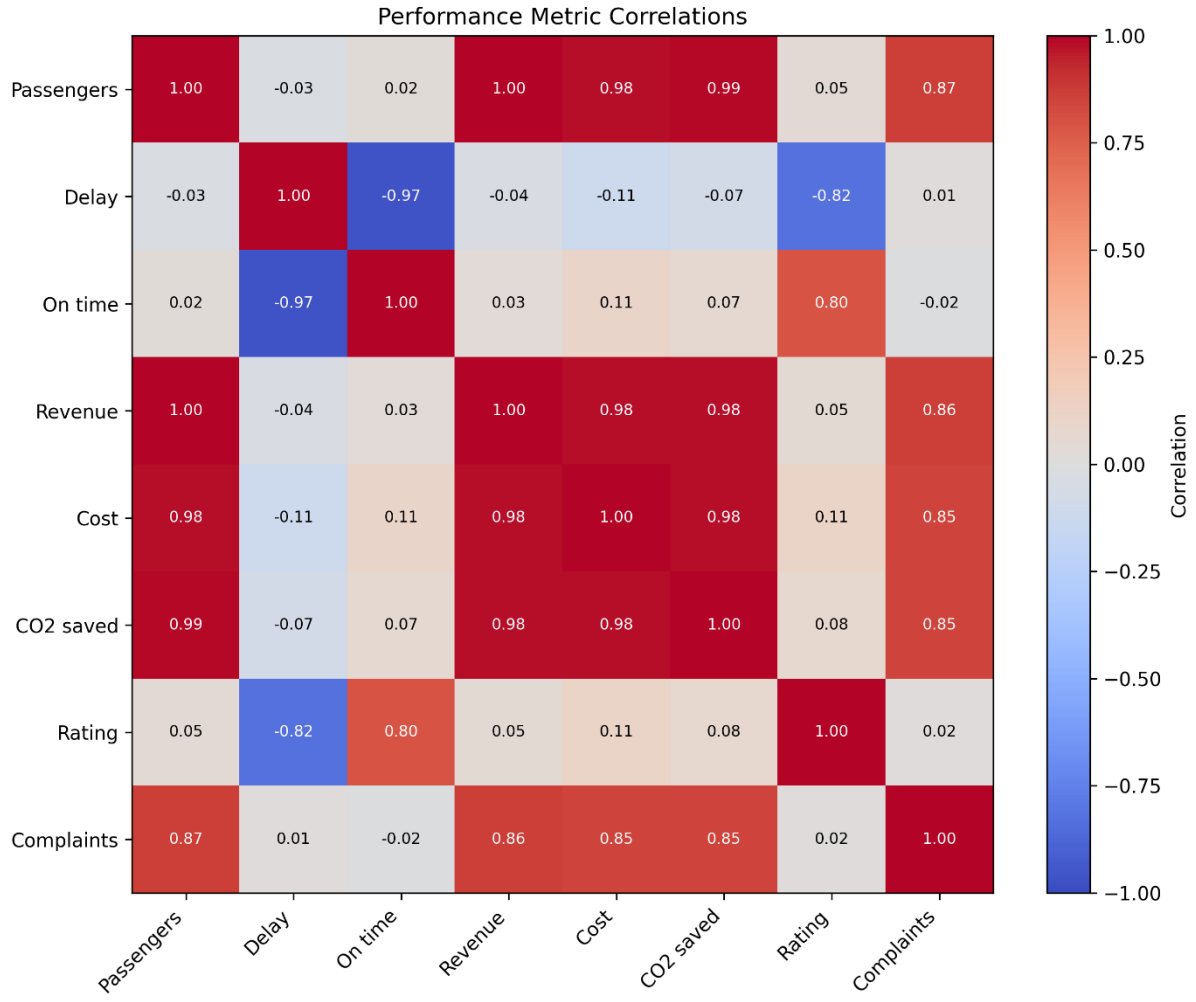


Figure 4: Performance metric correlations

There were differences in financial performance by mode. Metro generated approximately €73.33 million of revenue and a positive balance of approximately €4.30 million. The negative balances of Bike Share and E-Scooter were close to €5.60 million and €3.28 million respectively. The grouped bars allowed for the easy comparison of revenue and cost. The bubble plot was a merged accessibility, sustainability and demand plot. E-Scooter had the highest complaint rate at 18.36 per 10,000 passengers. Bike Share had the highest CO2 saving rate (182.81 kg per 1,000 passengers). Passenger demand was represented by the size of the bubble.

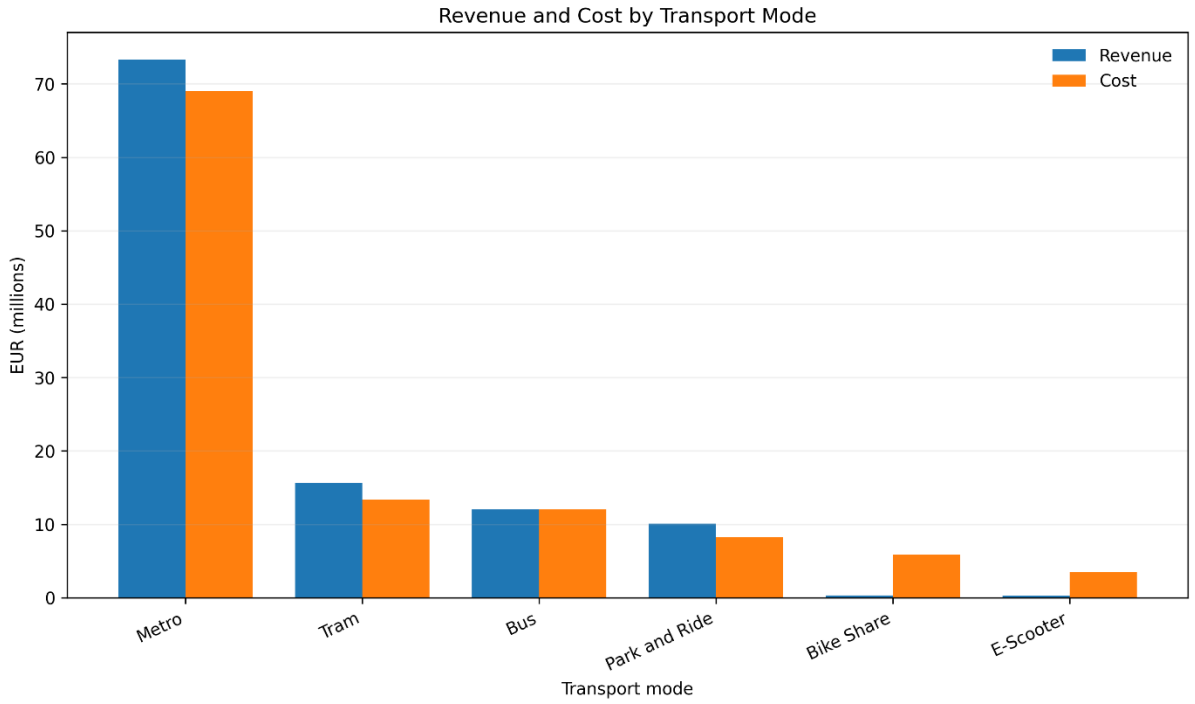


Figure 5: Revenue and cost by transport mode

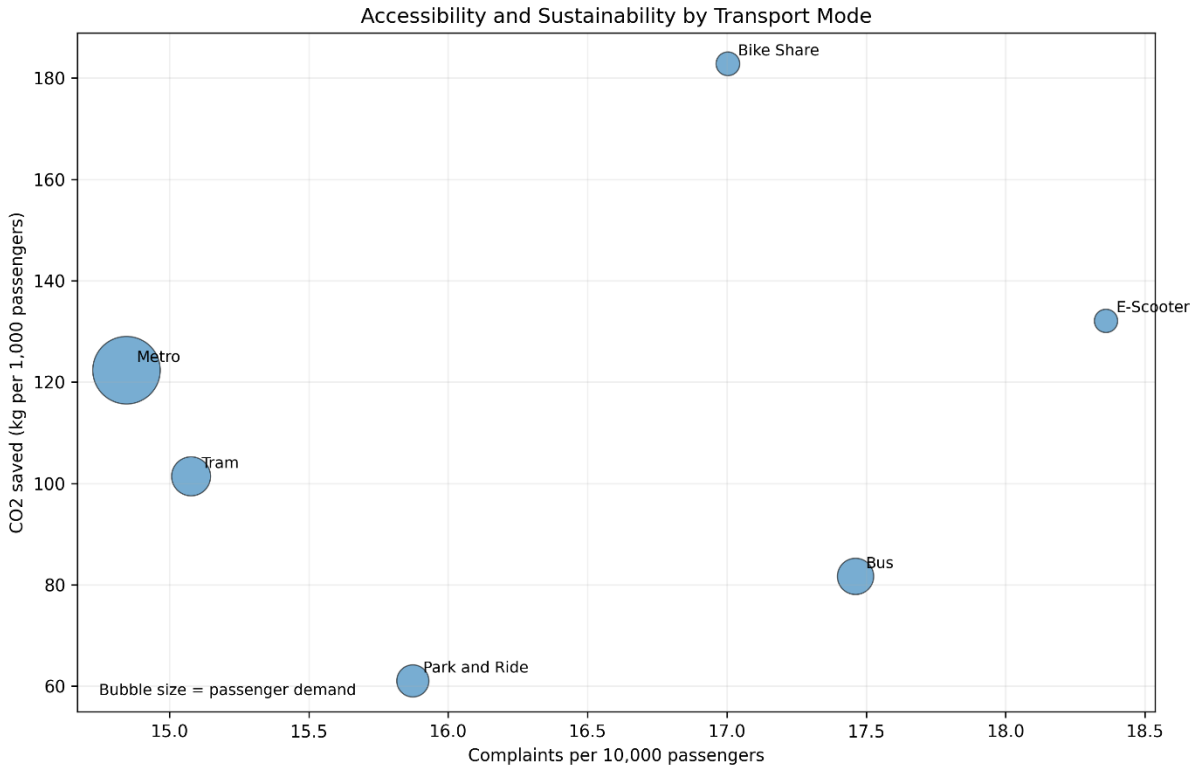


Figure 6: Accessibility and sustainability by transport mode

The visual forms were tailored to various users. Managers could scan information rapidly with the help of ranked bars and headline indicators. Numbers maintained their value for technical users. A consistent scale was used for the heatmap and the labels on the dashboard were matched with terms in the dataset. The selections enhanced clarity while not compromising on detail.

1.3. Interactive Dashboard

The Streamlit dashboard provided filters for date, country, city, mode, route, weather and event day. With the selected country, the city list was adjusted so that the selections were not unrelated. The views were accompanied by country/city and route data on demand, reliability, finance, rating, CO2, energy and complaints. Interactive dashboards can be used to facilitate organisation decisions by targeted exploration (Hjelle et al., 2024). Consistent indicators across locations and conditions (Expert Group for Urban Mobility, 2024), are also beneficial for sustainable mobility monitoring. The dashboard helped senior managers plan, provide reliability checks, improve accessibility, monitor sustainability and make investment decisions. The full network position was also shown on the dashboard prior to filtering. With this, managers had a benchmark to determine if the country, city, or mode of transport they selected had been more or less successful overall.

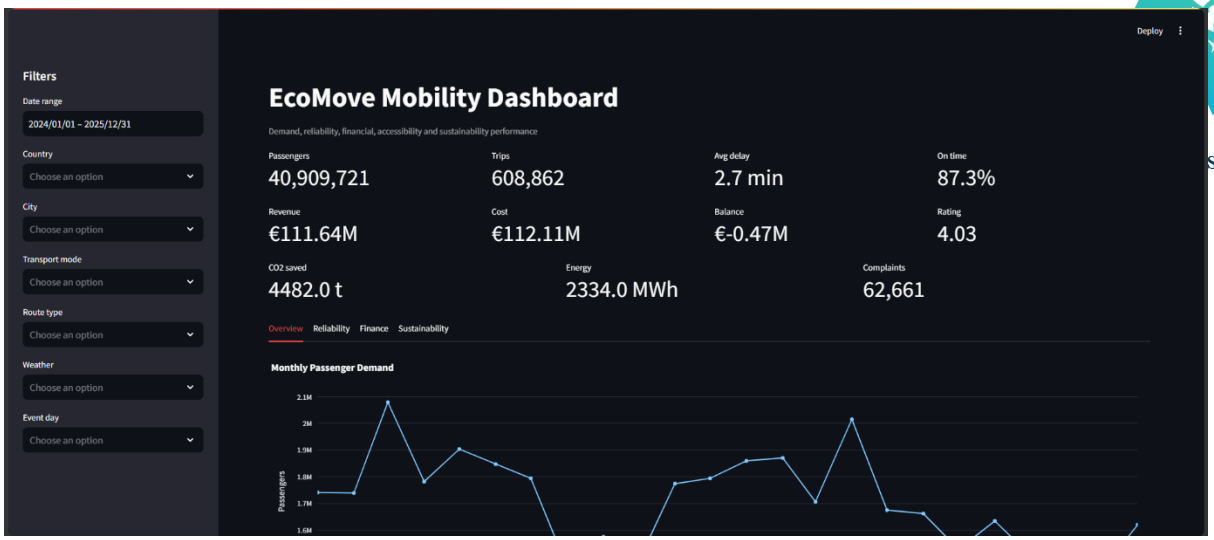


Figure 7: Full interactive dashboard

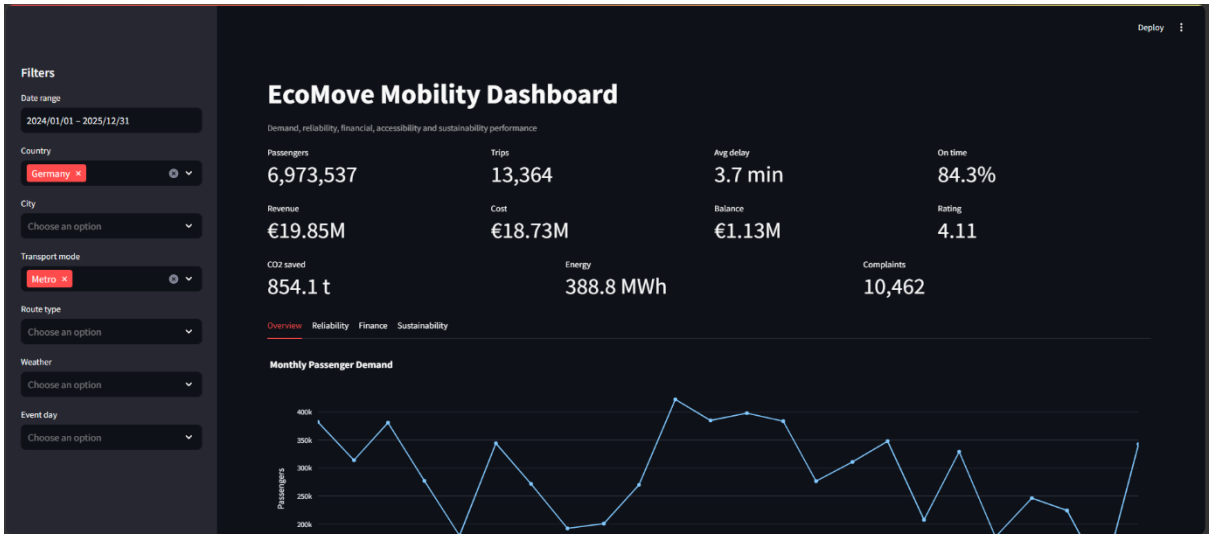


Figure 8: Filtered dashboard for Germany and Metro

Task 2 Data Visualisation Report

2.1. Critical Findings

There were time and place dependent and mode dependent variations between the different EcoMove performances. The highest numbers of requests were in Paris and Berlin, with lesser numbers in Lyon and Rotterdam. There was also month-on-month variation that made it clear that planning should not be based on one year. Following the International Transport Forum (2023), the performance of urban mobility is considered as a system problem that are related to demands, accessibility, emissions and mobility activities. Reliability was another concern. Delay was greater on event days than for all days within each weather group and the snow type caused the greatest delay. There was also a strong negative relationship between Delay and Customer Rating. These results are correlations and cannot be interpreted as causal effects of delay on ratings.

The financial performance was split. The Metro had the highest revenues and positive balance. The positive balances were achieved by Tram and Park and Ride, with Bus just slightly over break even. Bike Share and E-Scooter suffered significant losses. This does not imply that these modes are to be automatically deleted. The decision-making process around public transport and wider economic benefits includes considerations of service provision, as well as direct financial outcomes (International Association of Public Transport, 2024). Bike Share contributed the most CO2 savings, but incurred the largest negative balance. E-Scooter achieved a fairly high CO2 saving rate and the highest accessibility complaint rate. There has been a growing focus on environmental performance in European mobility assessment, in addition to service outcomes and sustainability objectives (European Environment Agency, 2026). A combined view is thus more beneficial than revenue-based views of modes. This is important because decision making in management of several outcomes simultaneously. It can be argued that for some of these purposes, a financially weaker mode may be able to provide access or environmental objectives or broader network reach that cannot be measured directly because of the transport system without relying solely on the revenue.

Table 2: Key findings

Area	Finding	Decision use
Demand	Paris and Berlin had the highest demand.	Review capacity.
Delay	Snow event days had the highest delays.	Plan for disruption.

Finance	Metro had the strongest balance.	Review investment.
Access	E-Scooter had the highest complaint rate.	Improve access.
CO2	Bike Share had the highest saving rate.	Support low-carbon planning.
Network	The overall balance was about minus €0.47M.	Review the portfolio.

2.2 Accessibility and Wider Implications

Accessibility should be considered as it is reflected in different complaint rates across modes. The lowest rate was for Metro at 14.85 per 10,000 passengers, and E-Scooter had a rate of 18.36. These results are used to identify priorities, but complaints don't explain why each user has a barrier. Accessibility may be related to the entire journey, from accessing and leaving services (Levine, 2024). Further analysis might include complaint data, road access, boarding conditions and passenger feedback. Environmental parameters are also of concern. The amount of CO2 saved was generally very tightly correlated with passenger numbers, thus the totals may be a reflection of scale. Sustainable mobility policies can affect emissions and energy demand, making both measures important (Winkler et al., 2023).

So, the management decisions should be made considering various objectives. A company may be financially successful, yet still have reliability or accessibility issues. If finances are not as robust, this can be combined with environmental gains or network coverage gains. These trade-offs are revealed in financial, service and environment indicators on the dashboard. Managers can compare if a mode has strong demand but weak punctuality, or if environmental benefits are enjoyed but accessibility outcomes are not. Also, transparency is important in public-sector investments, where decisions impact on both budgets and environmental targets, as well as on the passengers. However, the data does not provide demographic data, capacities of routes and purposes of journeys. These gaps limit conclusions about specific user groups and service conditions.

2.3 Dashboard Evaluation

The dashboard enabled managers to switch from a network perspective to focused operational perspectives without having to rely on separate reports. Filters included country, city, mode, route, weather and event day. This function was shown by selecting Germany and Metro. The filtered view kept the demand, reliability, finance, and environmental measures in order to compare the same. Technical users could view detailed values, and non-technical managers could view the headline indicators. The dashboard remains descriptive. It can't define need or provide a rationale for all relationships. Future analysis would be enhanced by the availability of route capacity details, timetable, and passenger survey/cost details.

2.4 Recommendations and Conclusion

Four actions follow from the findings. First, Paris and Berlin should have regular capacity review due to their large numbers of passengers. A flexible schedule for the months is suggested due to the variation in demand throughout the year. Second, it is important that plans are more focused for events on the day and for severe weather, particularly in snow and rain and heat wave conditions. Third, the financial, sustainability and accessibility measures for Bike Share and E-Scooter need to be considered together. Fourth, both reliability and customer rating are worth monitoring, as they were strongly related. Overall, there was no single measure of evaluating EcoMove. The combination of demand, reliability, finance, accessibility and sustainability helped create a more solid footing for both business, social and environmental decisions and public-sector decisions.

References

Delfini, L., Spahiu, B. and Vizzari, G. (2024) 'Visual analytics for sustainable mobility: usability evaluation and knowledge acquisition for Mobility-as-a-Service (MaaS) data exploration', *Digital*, 4(4), pp. 821–845. Available at: https://doi.org/10.3390/digital4040041

European Environment Agency (2026) *Sustainability of Europe's mobility systems 2025*. Web report no. 02/2025. Available at: https://www.eea.europa.eu/en/analysis/publications/sustainability-of-europes-mobility-systems-2025 (Accessed: 24 October 2026).

Expert Group for Urban Mobility (2024) *EGUM opinion on the sustainable urban mobility indicators: best practice on monitoring SUMP implementation, especially on defining and applying sustainable urban mobility indicators and data collection*. European Commission. Available at: https://transport.ec.europa.eu/document/download/cb890007-af95-46e8-8f9c-1a29c1efefac_en?filename=EGUM_SUMP_subgroup_SUMI_opinion.pdf (Accessed: 24 October 2026).

Hjelle, S. et al. (2024) 'Organizational decision making and analytics: an experimental study on dashboard visualizations', *Information & Management*, 61(6), article 104011. Available at: https://doi.org/10.1016/j.im.2024.104011

Institute for Transportation and Development Policy (2024) *The BRT Standard*. Available at: https://itdp.org/wp-content/uploads/2024/03/BRT-STANDARD_MAR-2024_ITDP.pdf (Accessed: 24 October 2026).

International Association of Public Transport (2024) *Global Economic Outlook 2024: taking the pulse of the public transport sector*. UITP. Available at: https://www.uitp.org/wp-content/uploads/sites/7/2025/08/20241008_Economic-Outlook-2024_WEB.pdf (Accessed: 24 October 2026).

International Transport Forum (2023) *ITF Transport Outlook 2023*. OECD Publishing. Available at: https://doi.org/10.1787/b6cc9ad5-en

Levine, K. (2024) 'The bus is accessible, but how do you get to the bus: first and last mile experiences of disabled transit riders', *Journal of Public Transportation*, 26, article 100086. Available at: https://doi.org/10.1016/j.jpubtr.2024.100086

Tsouros, I. et al. (2024) 'From raw data to informed decisions: the development of an online data repository and visualization dashboard for transportation data', *Procedia Computer Science*, 231, pp. 161–167. Available at: https://doi.org/10.1016/j.procs.2023.12.188

Winkler, L. et al. (2023) 'The effect of sustainable mobility transition policies on cumulative urban transport emissions and energy demand', *Nature Communications*, 14, article 2357. Available at: https://doi.org/10.1038/s41467-023-37728-x

Appendix

visual_01_time_trend.py

```python
def run(data=None):
    df = load_data() if data is None else data.copy()
    monthly = (
        df.assign(Month=df["Date"].dt.to_period("M").dt.to_timestamp())
        .groupby("Month", as_index=False)["Passengers"]
        .sum()
    )

    print("\nMonthly passenger demand")
    print(monthly.to_string(index=False))

    fig, ax = plt.subplots(figsize=(10, 5.5))
    ax.plot(monthly["Month"], monthly["Passengers"], marker="o", linewidth=1.8)
    ax.set_title("Monthly Passenger Demand")
    ax.set_xlabel("Month")
    ax.set_ylabel("Passengers")
    ax.set_xlim(monthly["Month"].min(), monthly["Month"].max())
    ax.xaxis.set_major_locator(mdates.MonthLocator(interval=3))
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%b\n%Y"))
    ax.yaxis.set_major_formatter(FuncFormatter(lambda value, _: f"{value / 1_000_000:.1f}M"))
    ax.grid(axis="y", alpha=0.25)
    fig.tight_layout()

    output = OUTPUT_DIR / "01_monthly_passenger_demand.png"
    fig.savefig(output, dpi=300, bbox_inches="tight")
    plt.close(fig)
    return output
```

Figure 9: Monthly demand code

visual_02_city_demand.py

```python
def run(data=None):
    df = load_data() if data is None else data.copy()
    city = (
        df.groupby("City", as_index=False)["Passengers"]
        .sum()
        .sort_values("Passengers", ascending=True)
    )

    print("\nPassenger demand by city")
    print(city.sort_values("Passengers", ascending=False).to_string(index=False))

    fig, ax = plt.subplots(figsize=(9, 6))
    ax.barh(city["City"], city["Passengers"])
    ax.set_title("Passenger Demand by City")
    ax.set_xlabel("Passengers")
    ax.set_ylabel("City")
    ax.xaxis.set_major_formatter(FuncFormatter(lambda value, _: f"{value / 1_000_000:.1f}M"))
    ax.grid(axis="x", alpha=0.2)
    fig.tight_layout()

    output = OUTPUT_DIR / "02_city_passenger_demand.png"
    fig.savefig(output, dpi=300, bbox_inches="tight")
    plt.close(fig)
    return output
```

Figure 10: City demand code

visual_03_delay_distribution.py

```python
def run(data=None):
    df = load_data() if data is None else data.copy()
    weather_order = ["Clear", "Cloudy", "Rain", "Windy", "Heatwave", "Snow"]
    event_order = ["No", "Yes"]
    colors = {"No": "#4C78A8", "Yes": "#F58518"}
    hatches = {"No": "", "Yes": "//"}

    summary = (
        df.groupby(["Weather", "Event_Day"], as_index=False)
        .agg(
            Avg_Delay=("Average_Delay_Minutes", "mean"),
            Median_Delay=("Average_Delay_Minutes", "median"),
        )
    )
    print("\nDelay by weather and event day")
    print(summary.round(2).to_string(index=False))

    positions = np.arange(len(weather_order))
    width = 0.32
    fig, ax = plt.subplots(figsize=(11, 6))

    for event in event_order:
        box_data = [
            df.loc[
                (df["Weather"] == weather) & (df["Event_Day"] == event),
                "Average_Delay_Minutes",
            ].dropna().values
            for weather in weather_order
        ]
        offset = -width / 2 if event == "No" else width / 2
        box = ax.boxplot(
            box_data,
            positions=positions + offset,
            widths=width * 0.85,
            patch_artist=True,
            showfliers=False,
            medianprops={"color": "black", "linewidth": 1.4},
```

Figure 11: Delay distribution code part 1

visual_03_delay_distribution.py

```
def run(data=None):
            whiskerprops={"color": "#444444", "linewidth": 1.0},
            capprops={"color": "#444444", "linewidth": 1.0},
        )

        for patch in box["boxes"]:
            patch.set_facecolor(colors[event])
            patch.set_edgecolor("#444444")
            patch.set_alpha(0.72)
            patch.set_hatch(hatches[event])

    ax.set_xticks(positions)
    ax.set_xticklabels(weather_order)
    ax.set_title("Delay Distribution by Weather and Event Day")
    ax.set_xlabel("Weather")
    ax.set_ylabel("Delay (minutes)")
    ax.legend(
        handles=[
            Patch(facecolor=colors["No"], edgecolor="#444444", label="Normal day", alpha=0.72),
            Patch(facecolor=colors["Yes"], edgecolor="#444444", hatch="//", label="Event day", alpha=0.72),
        ],
        frameon=False,
        loc="upper left",
    )
    ax.grid(axis="y", alpha=0.2)
    fig.tight_layout()

    output = OUTPUT_DIR / "03_delay_weather_event.png"
    fig.savefig(output, dpi=300, bbox_inches="tight")
    plt.close(fig)
    return output
```

Figure 12: Delay distribution code part 2

visual_04_correlation.py

```python
def run(data=None):
    df = load_data() if data is None else data.copy()
    columns = [
        "Passengers",
        "Average_Delay_Minutes",
        "On_Time_Percentage",
        "Ticket_Revenue_EUR",
        "Operating_Cost_EUR",
        "CO2_Saved_KG",
        "Customer_Rating",
        "Accessibility_Complaints",
    ]
    labels = [
        "Passengers",
        "Delay",
        "On time",
        "Revenue",
        "Cost",
        "CO2 saved",
        "Rating",
        "Complaints",
    ]
    corr = df[columns].corr()

    print("\nPerformance metric correlations")
    printable = corr.copy()
    printable.index = labels
    printable.columns = labels
    print(printable.round(2).to_string())
```

Figure 13: Correlation code part 1

visual_04_correlation.py

```python
def run(data=None):

    fig, ax = plt.subplots(figsize=(9, 7.5))
    image = ax.imshow(corr, vmin=-1, vmax=1, cmap="coolwarm", aspect="auto")
    colorbar = fig.colorbar(image, ax=ax)
    colorbar.set_label("Correlation")
    ax.set_xticks(range(len(labels)))
    ax.set_xticklabels(labels, rotation=45, ha="right")
    ax.set_yticks(range(len(labels)))
    ax.set_yticklabels(labels)
    ax.set_title("Performance Metric Correlations")

    for i in range(len(labels)):
        for j in range(len(labels)):
            value = corr.iloc[i, j]
            text_color = "white" if abs(value) >= 0.55 else "black"
            ax.text(j, i, f"{value:.2f}", ha="center", va="center", fontsize=8, color=text_color)

    fig.tight_layout()
    output = OUTPUT_DIR / "04_performance_correlations.png"
    fig.savefig(output, dpi=300, bbox_inches="tight")
    plt.close(fig)
    return output
```

Figure 14: Correlation code part 2

visual_05_financial_performance.py

```python
def run(data=None):
    df = load_data() if data is None else data.copy()
    finance = (
        df.groupby("Transport_Mode", as_index=False)
        .agg(
            Revenue=("Ticket_Revenue_EUR", "sum"),
            Cost=("Operating_Cost_EUR", "sum"),
        )
        .sort_values("Revenue", ascending=False)
    )
    finance["Balance"] = finance["Revenue"] - finance["Cost"]

    print("\nFinancial performance by transport mode")
    print(finance.round(2).to_string(index=False))

    x = np.arange(len(finance))
    width = 0.38
    fig, ax = plt.subplots(figsize=(10, 6))
    ax.bar(x - width / 2, finance["Revenue"] / 1_000_000, width, label="Revenue")
    ax.bar(x + width / 2, finance["Cost"] / 1_000_000, width, label="Cost")
    ax.set_xticks(x)
    ax.set_xticklabels(finance["Transport_Mode"], rotation=25, ha="right")
    ax.set_title("Revenue and Cost by Transport Mode")
    ax.set_xlabel("Transport mode")
    ax.set_ylabel("EUR (millions)")
    ax.legend(frameon=False)
    ax.grid(axis="y", alpha=0.2)
    fig.tight_layout()

    output = OUTPUT_DIR / "05_mode_revenue_cost.png"
    fig.savefig(output, dpi=300, bbox_inches="tight")
    plt.close(fig)
    return output
```

Figure 15: Financial performance code

```python
def run(data=None):
    df = load_data() if data is None else data.copy()
    mode = (
        df.groupby("Transport_Mode", as_index=False)
        .agg(
            Passengers=("Passengers", "sum"),
            Complaints=("Accessibility_Complaints", "sum"),
            CO2=("CO2_Saved_KG", "sum"),
            Energy=("Energy_Consumption_KWh", "sum"),
        )
    )
    mode["Complaints_per_10k"] = mode["Complaints"] / mode["Passengers"] * 10_000
    mode["CO2_per_1000"] = mode["CO2"] / mode["Passengers"] * 1_000
    mode["Energy_per_1000"] = mode["Energy"] / mode["Passengers"] * 1_000

    print("\nAccessibility and sustainability by transport mode")
    print(
        mode[
            ["Transport_Mode", "Passengers", "Complaints_per_10k", "CO2_per_1000", "Energy_per_1000"]
        ].round(2).to_string(index=False)
    )

    bubble_size = 180 + (mode["Passengers"] / mode["Passengers"].max()) * 1400
    fig, ax = plt.subplots(figsize=(10, 6.5))
    ax.scatter(
        mode["Complaints_per_10k"],
        mode["CO2_per_1000"],
        s=bubble_size,
        alpha=0.6,
        edgecolors="black",
```

Figure 16: Accessibility and sustainability code part 1

```python
def run(data=None):
        linewidths=0.7,
    )

    for _, row in mode.iterrows():
        ax.annotate(
            row["Transport_Mode"],
            (row["Complaints_per_10k"], row["CO2_per_1000"]),
            xytext=(6, 5),
            textcoords="offset points",
            fontsize=9,
        )

    ax.set_title("Accessibility and Sustainability by Transport Mode")
    ax.text(
        0.02,
        0.02,
        "Bubble size = passenger demand",
        transform=ax.transAxes,
        fontsize=9,
        va="bottom",
    )
    ax.set_xlabel("Complaints per 10,000 passengers")
    ax.set_ylabel("CO2 saved (kg per 1,000 passengers)")
    ax.grid(alpha=0.2)
    fig.tight_layout()

    output = OUTPUT_DIR / "06_mode_accessibility_sustainability.png"
    fig.savefig(output, dpi=300, bbox_inches="tight")
    plt.close(fig)
    return output
```

Figure 17: Accessibility and sustainability code part 2

dashboard.py

```python
st.sidebar.header("Filters")

start_date = df["Date"].min().date()
end_date = df["Date"].max().date()
date_range = st.sidebar.date_input(
    "Date range",
    value=(start_date, end_date),
    min_value=start_date,
    max_value=end_date,
)

countries = st.sidebar.multiselect("Country", sorted(df["Country"].unique()))
city_source = df[df["Country"].isin(countries)] if countries else df
cities = st.sidebar.multiselect("City", sorted(city_source["City"].unique()))
modes = st.sidebar.multiselect("Transport mode", sorted(df["Transport_Mode"].unique()))
routes = st.sidebar.multiselect("Route type", sorted(df["Route_Type"].unique()))
weather = st.sidebar.multiselect("Weather", sorted(df["Weather"].unique()))
events = st.sidebar.multiselect("Event day", sorted(df["Event_Day"].unique()))

filtered = df.copy()
if isinstance(date_range, (tuple, list)) and len(date_range) == 2:
    filtered = filtered[
        (filtered["Date"].dt.date >= date_range[0])
        & (filtered["Date"].dt.date <= date_range[1])
    ]
filtered = apply_filter(filtered, "Country", countries)
filtered = apply_filter(filtered, "City", cities)
filtered = apply_filter(filtered, "Transport_Mode", modes)
filtered = apply_filter(filtered, "Route_Type", routes)
filtered = apply_filter(filtered, "Weather", weather)
```

Figure 18: Dashboard filters code

```python
filtered = apply_filter(filtered, "Event_Day", events)

if filtered.empty:
    st.warning("No records match the selected filters.")
    st.stop()

revenue = filtered["Ticket_Revenue_EUR"].sum()
cost = filtered["Operating_Cost_EUR"].sum()
balance = revenue - cost
avg_delay = weighted_mean(filtered, "Average_Delay_Minutes", "Trips")
on_time = weighted_mean(filtered, "On_Time_Percentage", "Trips")
rating = weighted_mean(filtered, "Customer_Rating", "Passengers")

row1 = st.columns(4)
row1[0].metric("Passengers", f"{filtered['Passengers'].sum():,.0f}")
row1[1].metric("Trips", f"{filtered['Trips'].sum():,.0f}")
row1[2].metric("Avg delay", f"{avg_delay:.1f} min")
row1[3].metric("On time", f"{on_time:.1f}%")

row2 = st.columns(4)
row2[0].metric("Revenue", f"€{revenue / 1_000_000:.2f}M")
row2[1].metric("Cost", f"€{cost / 1_000_000:.2f}M")
row2[2].metric("Balance", f"€{balance / 1_000_000:.2f}M")
row2[3].metric("Rating", f"{rating:.2f}")

row3 = st.columns(3)
row3[0].metric("CO2 saved", f"{filtered['CO2_Saved_KG'].sum() / 1_000:.1f} t")
row3[1].metric("Energy", f"{filtered['Energy_Consumption_KWh'].sum() / 1_000:.1f} MWh")
row3[2].metric("Complaints", f"{filtered['Accessibility_Complaints'].sum():,.0f}")

overview_tab, reliability_tab, finance_tab, sustainability_tab = st.tabs(
    ["Overview", "Reliability", "Finance", "Sustainability"]
)
```

Figure 19: Dashboard KPI code

```python
with overview_tab:
    monthly = (
        filtered.assign(Month=filtered["Date"].dt.to_period("M").dt.to_timestamp())
        .groupby("Month", as_index=False)["Passengers"]
        .sum()
    )
    fig = px.line(monthly, x="Month", y="Passengers", markers=True, title="Monthly Passenger Demand")
    fig.update_xaxes(range=[monthly["Month"].min(), monthly["Month"].max()])
    fig.update_yaxes(tickformat="~s")
    st.plotly_chart(clean_figure(fig), use_container_width=True)

    left, right = st.columns(2)
    country = filtered.groupby("Country", as_index=False)["Passengers"].sum().sort_values("Passengers")
    fig = px.bar(country, x="Passengers", y="Country", orientation="h", title="Passenger Demand by Country")
    fig.update_xaxes(tickformat="~s")
    left.plotly_chart(clean_figure(fig), use_container_width=True)

    city = filtered.groupby("City", as_index=False)["Passengers"].sum().sort_values("Passengers")
    fig = px.bar(city, x="Passengers", y="City", orientation="h", title="Passenger Demand by City")
    fig.update_xaxes(tickformat="~s")
    right.plotly_chart(clean_figure(fig), use_container_width=True)

    route = filtered.groupby("Route_Type", as_index=False)["Passengers"].sum().sort_values("Passengers")
    fig = px.bar(
        route,
        x="Passengers",
        y="Route_Type",
        orientation="h",
        title="Passenger Demand by Route Type",
        labels={"Route_Type": "Route type"},
    )
    fig.update_xaxes(tickformat="~s")
    st.plotly_chart(clean_figure(fig), use_container_width=True)
```

Figure 20: Dashboard interactive views code

Github

https://github.com/ankit-vadadoriya/Data-Visualisation